Name: Sam Setta
Date: May 17, 2026
Course: Foundations Of Python Programming
Assignment: Assignment 05

Using dictionaries and structured error handling

Introduction

This document outlines how to use dictionaries to store student data with key-value pairs and save to a JSON file, while including structured error handling to capture user errors. The starter script included the program code from module 04 and was edited to include these features (dictionaries, error handling).

Creating a Python script for the program

Opening python script in PyCharm and editing the header

Similar to module 04, a new project was created in the module 05 directory before opening the starter script (Assignment05-starter.py), saving to Assignment05.py and editing the header with my name and date.

Loading modules

For the first time in this course, a module needed to be loaded before running the program because this script works with java script notation files (JSON). To do so, the import() function is used to load the JSON module, which allows users to load, read, and write JSON-formatted data (like dictionaries) to and from JSON files.

Define the Data Constants and Variables

The only data constant that needed to change was the FILENAME constant, which was changed from a .csv extension to .json to save the data into a JSON-formatted final file (Enrollment.json).

While the data variables remained the same, the student_data variable was set to an empty dictionary with curly brackets { }, instead of the list [ ] it was set to in the last module.

Read the file data into a dictionary

To load the data from the JSON-formatted data file, with the open() function and read option, before using the imported json module's function json.load() to load the data into

the students dictionary. After reading the data into a dictionary, to enable adding more student data based on user input, the file is closed with the close() function.

Main body of the script

The structure of the python program is the same as module 04. Briefly, this program uses a while statement to prompt menu option entry after each option is finished collecting user input or printing user output (to the user display or saved to a file), until the break statement is encountered for menu option 4.

Input student data (option 1)

Option 1 prompts user entry for student data (first, last and course name), the data is saved to the respective data variables (student_first, student_last, course_name). Unlike module 04's program, the student data variables are saved to a dictionary with the respective keys of FirstName, LastName, CourseName for each value (see below). This allows the data values to be pulled from the student_data dictionary to which they are saved by calling the key. The new student data dictionary is also appended to the students list variable to save for later data processing.

```
# add to student_data
student_data = {"FirstName": student_first_name,
                "LastName": student_last_name,
                "CourseName": course_name}
# append to students list data
students.append(student_data)
# inform user of data entry
print(f"You have registered {student_data["FirstName"]}
{student_data["LastName"]} for {student_data["CourseName"]}.")
```

To inform the user that the student data was entered, a print statement is also included that pulls the respective value from each key (see above), taking advantage of the dictionaries key-value storage.

Present the current data (option 2)

The code to present current data used if menu option 2, prints the student first name, last name and course name separated by commas for each dictionary in the students list (code below). Values for each are pulled from the dictionary using keys, like that used in the print statement for option 1. Otherwise, the code here, is the same as the module 04 program, printing a line of 50 dashes, before displaying the name for each value that will be printed from the for loop through each row of the list of students before printing a final line of 50 dashes, and using a continue statement to once again print menu options.

```
# Process the data to create and display a custom message
print("-"*50) # make the print out look nice
print("First name, Last name, Course name:")
for student in students:
    print(f"{student["FirstName"]}, {student["LastName"]},
{student["CourseName"]}")
print("-"*50)  # make the print out look nice
continue
```

Save data to a JSON data file (option 3)

To save the list of dictionaries to a json file, using the json module, the code is much simpler than restructuring into a CSV format and saving. The file (Enrollment.json) file is opened with the open function in write mode, json.dump() is then used to save the students data to the Enrollment.json file (code below). The indent option is used to make the saved file more readable, and the file is closed with the close() function, before a message is printed out that the file was saved (see code below).

```
file = open(FILE_NAME, "w")
json.dump(students, file, indent=2)
file.close()
print("Data Saved!")
```

Exit the program (option 4)

To save the list. If menu option 4 is chosen, the script is the same as the program for the assignment from module 04. The break statement is used to break the while loop.

The user input wasn't one of the four options

Finally, if any other options are chosen by the user, the program asks the reader to enter one of the available options before printing the menu once again, using the else statement, and print() function.

Structured error handling

Loading file into script

When loading the data in from a JSON file, structured error handling is used with try-except to ensure the file exists and the program continues even if unable to load data into the students dictionary. This is accomplished by first using try with json.load() as outlined above, if this is successfully done, the program moves to the while statement and prints the menu (see code below). If the file is not found, an except statement is included that captures the FileNotFoundError, prints a custom message that file must exist before running and printing the technical error message from python, along with the type of error, separated by a new line (new line character) (see code below). If the

error is not that the file wasn't found, the next expect statement includes all other errors, with the same custom and technical details for the error separated by new lines (see code below). A finally block is used at the end of the try except error handling to catch any instances where a file is found and opened, but some error occurred before it was closed, the if statement checks if the file exits but is still open (file.closed == False), and uses the close() function if so (code below).

```
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()
except FileNotFoundError as e:
    print("Text file must exist before running this script!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
    print("There was a non-specific error!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
finally:
    if file is not None and file.closed == False:
        file.close()
```

User entry of student data

A try-exception is once again used for the student data entry in option 1, the main body of the option is written within the first try block. After both the student first name and last name is to the user input with the input function the code immediately following checks if there are any non-alphabetic entries for either by using the isalpha() function, if so a ValueError is raised using the code raise ValueError with a custom message (see below).

```
if not student_first_name.isalpha():
    raise ValueError("Student first name should not contain numbers.")

if not student_last_name.isalpha():
    raise ValueError("Student last name should not contain numbers.")
```

The except statements associated with the user data entry outlined here are either because of a ValueError or Exception. The ValueError is captured by the earlier if not statement and raise ValueError, the custom message is reported by the string representation of the message (e.__str__()) in the following code. The docstring technical error (e.__doc__) is also included in the error message to give more details to the user.

```
except ValueError as e:
    print("-- Technical Error Message -- ")
    print(e.__doc__)
    print(e.__str__())  # Prints the custom message
```

Any other remaining error type besides ValueError is caught within the Exception class, with an error message reported by the docstring (see below).

```
except Exception as e:
    print("There was a non-specific error!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
```

Saving data to a JSON data file (option 3)

The final structured error handling within this data program is for saving data to the JSON file. Using similar try-except error handling as reading the JSON file, the code opening the file 'Enrollment.json' in write mode and saving the data using json.dump() is within the try code block (see below).

```
except TypeError as e:
    print("Please check that the data is a valid JSON format\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
```

A TypeError captures errors that aren't formatted properly to save into the json file, printing the message that 'the data must be in JSON format'. The second except block is the same code chunk used for the student data entry code, capturing all other exceptions from trying to save the JSON file. A finally code block is used to ensure the file is closed if it was opened and ran into an error, similar to the code used to ensure the file was closed after reading data at the beginning of the script (see below).

```
finally:
    if file is not None and file.closed == False:
        file.close()
```

Saving script in PyCharm

After finishing the script, I saved the file to the same directory and filename I saved it as at the start of the script (Documents/UW_Spr26_PythonCourse/Module05/Assignment/) by going to File > Save All while in PyCharm.

Testing the program

Testing program in PyCharm and the command line

I tested the program in both PyCharm and command line. While working in PyCharm, I was already in the module 05 project, but after opening command line I first navigated to my python course directory and module 05 directory within Terminal in MacOS. I ran the program code in in python with the following code: `python3 Assignment04.py`. I tested the program in both PyCharm and command line with the same steps that are outlined below but included the different output of each from each program in screenshots.

To check the program would output the expected error message when loading in the JSON file if the file is not found, I renamed the FILENAME to 'Enrollment.jsonWRONGFILE' before running the program and received the expected error message. After the error message was displayed to the user, the menu was still reported as expected, since the json.load() function was executed within the try-except block (Figure 1). The FILENAME was changed back to the original, 'Enrollment.json', before restarting the program in both PyCharm and command-line.

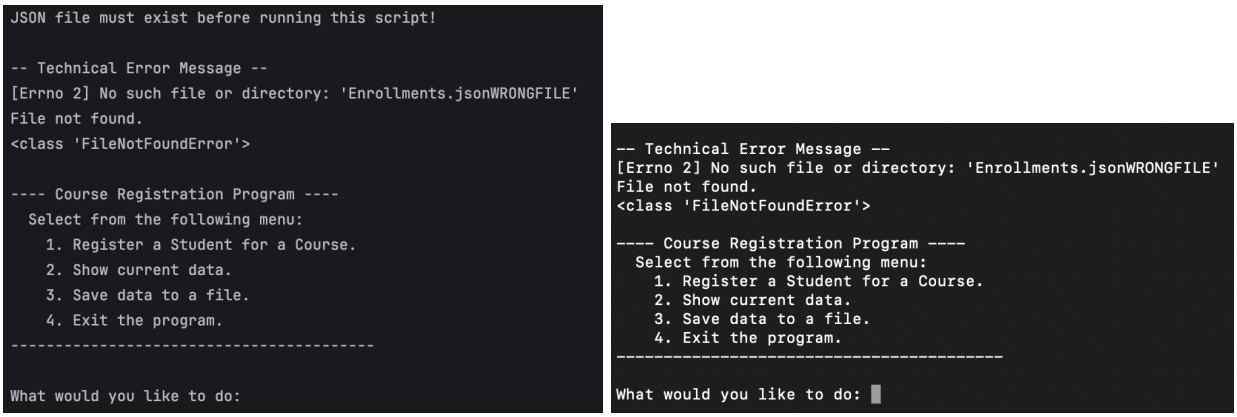

Figure 1. Screenshot of the error printed when attempting to load in a file that doesn't exist (Enrollment.jsonWRONGFILE) in PyCharm (left) and command-line (right).

To test the program takes user input, I started the program and input option 1, where I entered student registration information (first, last and course name). Afterwards I selected option 2, which displayed the data from the data file and the new entry (Vic, Vu, Python201), before selecting option 3, and getting the expected, 'Data Saved!', message (Figure 2). Finally, I selected option 4 to exit the program, which exited as expected.

```
What would you like to do: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python201
You have registered Vic Vu for Python201.

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 2
------------------------------------------------
First name, Last name, Course name:
Bob, Smith, Python 100
Sue, Jones, Python 100
Vic, Vu, Python201
------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 3
Data Saved!

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do:
```

```
What would you like to do: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python201
You have registered Vic Vu for Python201.

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 2
------------------------------------------------
First name, Last name, Course name:
Bob, Smith, Python 100
Sue, Jones, Python 100
Vic, Vu, Python201
Vic, Vu, Python201
------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 3
Data Saved!
```

Figure 2. Screenshot of user entry when selecting option 1, present the current data including the newly entered information, and saving the data to a file in PyCharm (left) and command-line (right).

To check the user input was saved properly, the Enrollment.json file was opened in both PyCharm and visual studio code (VS code) (Figure 3). Both had the expected user input, with Vic Vu twice, since this student's registration was entered in both PyCharm and command-line (Figure 3). The JSON file was also indented as expected based on the index=2 option added to the json.dump() function (Figure 3).

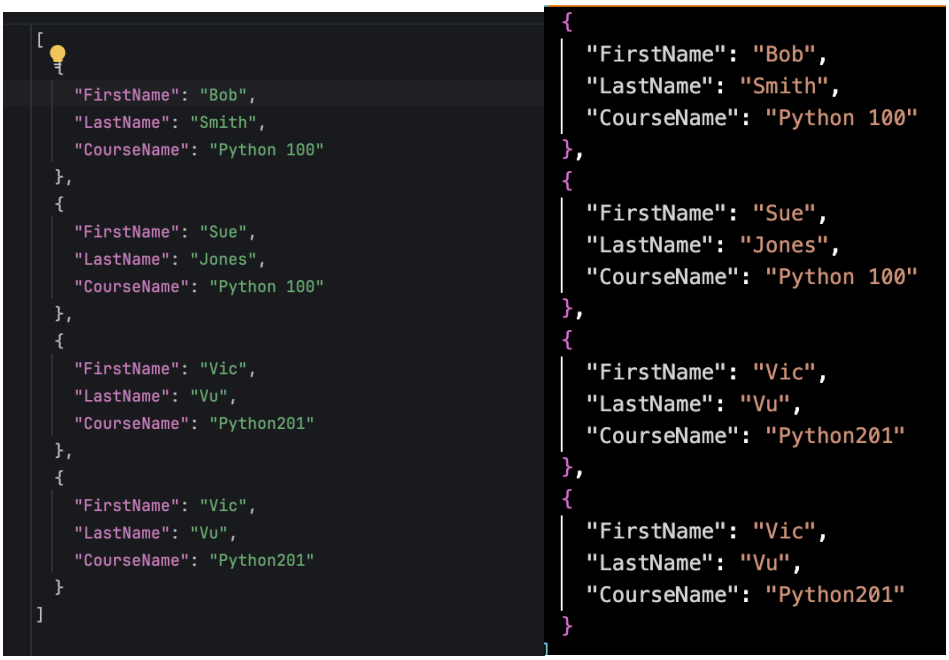

Figure 3. Screenshot of Enrollment.json file after saving new entry in PyCharm (left) and VS code (right).

To test if the program allows multiple registrations from user input and that these entries were saved to the JSON file, I restarted the program in both PyCharm and command-line. I selected option 1, and added a new student name and course name, before repeating the process to ensure I could add multiple entries (Figure 4). I checked the new data would be presented to the user, I selected option 2 and data was printed as expected, before choosing option 3 which notified me that the data was saved (Figure 4). To double-check the file data was saved properly, I re-opened the Enrollment.json file in both PyCharm and VS code and saw the newly entered data in the file (Figure 5).

```
What would you like to do: 1
Enter the student's first name: Purple
Enter the student's last name: PeopleEater
Please enter the name of the course: Monsters101
You have registered Purple PeopleEater for Monsters1

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 1
Enter the student's first name: Rosalind
Enter the student's last name: Franklin
Please enter the name of the course: Genetics101
You have registered Rosalind Franklin for Genetics10

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 2
------------------------------------------------
First name, Last name, Course name:
Bob, Smith, Python 100
Sue, Jones, Python 100
Vic, Vu, Python201
Vic, Vu, Python201
Purple, PeopleEater, Monsters101
Rosalind, Franklin, Genetics101
------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 3
Data Saved!

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
```

```
What would you like to do: 1
Enter the student's first name: Adrianne
Enter the student's last name: Lenker
Please enter the name of the course: Music101
You have registered Adrianne Lenker for Music101.

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 1
Enter the student's first name: Stevie
Enter the student's last name: Wonder
Please enter the name of the course: Music101
You have registered Stevie Wonder for Music101.

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 2
------------------------------------------------
First name, Last name, Course name:
Bob, Smith, Python 100
Sue, Jones, Python 100
Vic, Vu, Python201
Vic, Vu, Python201
Purple, PeopleEater, Monsters101
Rosalind, Franklin, Genetics101
Adrianne, Lenker, Music101
Stevie, Wonder, Music101
------------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------

What would you like to do: 3
Data Saved!
```

Figure 4. Screenshot of program testing to check if multiple student data entries code be entered and saved in PyCharm (left) and command-line (right).

Figure 5. Screenshot of Enrollment.json file after saving multiple new entries in PyCharm (left) and VS code (right).

Finally, I re-opened the program once again to check I would get the expected eror message if attempting to enter a first name or last name with non-alphabetic characters (Figure 6). I received the expected error warning that the names should not contain numbers before the menu options were once again displayed (Figure 6).

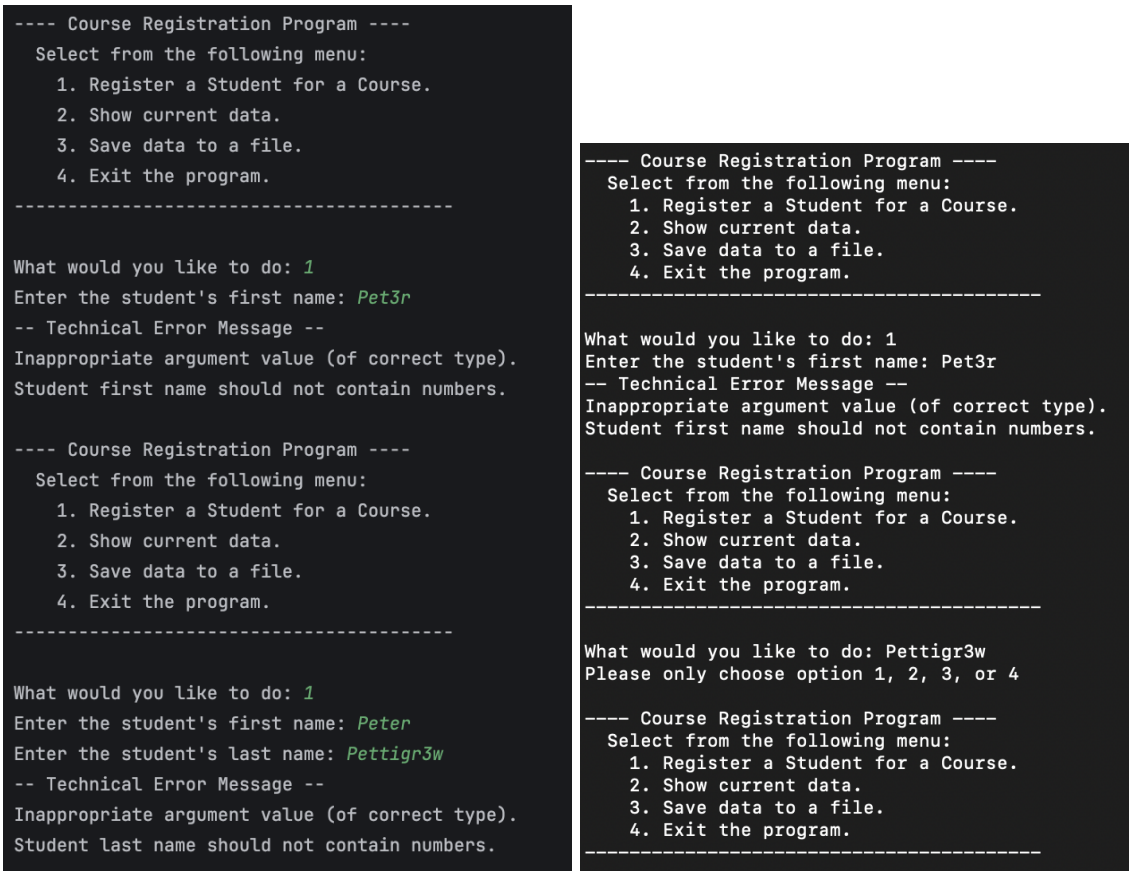

Figure 6. Screenshot of program testing to check if errors are presented to the user when numbers are used for student names (either first or last) in PyCharm (left) and command-line (right).

Summary

In this document, I've outlined how to improve upon the Assignment04.py program by saving data as dictionaries that include key-value pairs and including try-except blocks to help with defensive error handling that might result from unexpected user entries. The program cycles through multiple options to take in new user input for student registration, provides the option to display the registered students, save the data, and exit the program. Important concepts from Module 05 were used, including creating dictionaries, opening and saving to JSON data files, and using try-except blocks.